OOP Interview Questions

Complete guide for Junior .NET Developers — All concepts, all answers

Topic	Level	Questions
Classes & Objects	Basic	5
Encapsulation	Basic	4
Inheritance	Intermediate	6
Polymorphism	Intermediate	5
Abstraction	Basic	4
Interfaces & Design Principles	Intermediate	4
Advanced OOP	Advanced	7

Classes & Objects — Basic

Q1. What is a class and what is an object? How are they different?

Answer: A class is a blueprint/template that defines properties (fields) and behaviors (methods). An object is a concrete instance of a class created with new. Example: Car is a class; myCar = new Car() is an object.

Tags: class • object • instance

Q2. What are the differences between a value type and a reference type in C#?

Answer: Value types (int, struct, bool) are stored on the stack and hold data directly. Reference types (class, string, array) are stored on the heap and the variable holds a memory address. Assignment copies value for value types but copies the reference for reference types.

Tags: value type • reference type • stack • heap

Q3. What is the difference between a class and a struct in C#?

Answer: Classes are reference types; structs are value types. Structs cannot have parameterless constructors (pre-.NET 6), don't support inheritance, and are copied on assignment. Use structs for small, immutable data (e.g. Point, DateTime).

Tags: class • struct

Q4. What are constructors? What is constructor overloading?

Answer: A constructor is a special method called when an object is created. Constructor overloading means having multiple constructors with different parameter lists so objects can be created in different ways.

Tags: constructor • overloading

Q5. What is the difference between a static class and a non-static class?

Answer: A static class cannot be instantiated and can only have static members. It is used for utility methods (e.g. Math, Console). A non-static class can be instantiated and have both static and instance members.

Tags: static • instance

Encapsulation — Basic

Q6. What is encapsulation and why is it important?

Answer: Encapsulation bundles data and methods together and restricts direct access to internal state. It protects data integrity and hides implementation details. In C# it is achieved with access modifiers (private, public, etc.) and properties.

Tags: encapsulation • access modifiers

Q7. What are access modifiers in C# and what does each one do?

Answer: public: accessible anywhere. private: only within the same class. protected: within the class and subclasses. internal: within the same assembly. protected internal and private protected are combinations of the above.

Tags: public • private • protected • internal

Q8. What is the difference between a field and a property in C#?

Answer: A field is a variable declared directly in a class. A property exposes a field via get/set accessors, allowing validation or logic. Auto-properties (public int Age { get; set; }) auto-generate a backing field.

Tags: field • property • getter • setter

Q9. What is a readonly field vs a const field?

Answer: const is a compile-time constant; its value must be known at compile time. readonly is a runtime constant; it can be set in the constructor and is stored per instance. Use readonly for values that depend on runtime data.

Tags: readonly • const

Inheritance — Intermediate

Q10. What is inheritance and what problem does it solve?

Answer: Inheritance lets a derived class acquire members of a base class using ':'. It promotes code reuse and models 'is-a' relationships. Example: Dog : Animal.

Tags: inheritance • base class • derived class

Q11. What is the difference between method overriding and method hiding?

Answer: Overriding uses virtual/override keywords and enables polymorphism — the derived method is called through a base reference. Hiding uses the new keyword; the base method is hidden but not replaced, so calling through a base reference still calls the base version.

Tags: override • new • virtual

Q12. What does the 'sealed' keyword do?

Answer: sealed on a class prevents it from being inherited. On a method, it prevents further overriding in derived classes. For example, the string class in .NET is sealed.

Tags: sealed • inheritance

Q13. Can a class inherit from multiple classes in C#? How is this handled?

Answer: No, C# does not support multiple class inheritance (to avoid the diamond problem). However, a class can implement multiple interfaces, which achieves similar flexibility without ambiguity.

Tags: multiple inheritance • interface

Q14. What is the role of the 'base' keyword?

Answer: base refers to the parent class. It is used to call the parent constructor (base()) or a parent method that has been overridden (base.MethodName()).

Tags: base • constructor chaining

Q15. What is constructor chaining in C#?

Answer: Constructor chaining calls one constructor from another using this() (same class) or base() (parent class) to avoid code duplication. Example: public Car() : this("Red") { }

Tags: constructor • this • base

Polymorphism

Intermediate

Q16. What is polymorphism? What are its two main types?

Answer: Polymorphism allows objects of different types to be treated through a common interface. Compile-time polymorphism: method overloading and operator overloading. Runtime polymorphism: method overriding with virtual/override.

Tags: polymorphism • compile-time • runtime

Q17. What is the difference between method overloading and method overriding?

Answer: Overloading: same method name, different parameter list, resolved at compile time (same class). Overriding: same signature in parent and child class, resolved at runtime using virtual/override.

Tags: overloading • overriding

Q18. What is dynamic dispatch (late binding)?

Answer: Dynamic dispatch means the method called is determined at runtime based on the actual object type, not the declared type. This happens when you call a virtual/override method through a base class reference.

Tags: dynamic dispatch • virtual • vtable

Q19. What is an abstract class? When would you use it?

Answer: An abstract class cannot be instantiated and may contain abstract methods (no body) that derived classes must implement. Use it when classes share common behavior but each must define their own version of certain methods.

Tags: abstract • abstract class

Q20. What is the difference between an abstract class and an interface?

Answer: Abstract class: can have state (fields), constructors, and implemented methods. Single inheritance only. Interface: no state (before C# 8), no constructors, all members public by default. Multiple interfaces can be implemented. Use abstract class for shared base behavior, interface for

defining a capability contract.

Tags: abstract • interface

Abstraction — Basic

Q21. What is abstraction in OOP?

Answer: Abstraction hides complex implementation details and exposes only what is necessary. In C# it is achieved through abstract classes and interfaces. For example, you can call Drive() on a Car without knowing how the engine works internally.

Tags: abstraction

Q22. What is an interface in C# and how do you define one?

Answer: An interface defines a contract — a set of method/property signatures that implementing classes must fulfill. Defined with the interface keyword. Example: interface IShape { double Area(); }. A class implements it with : IShape.

Tags: interface • contract

Q23. Can an interface have a default implementation in C#?

Answer: Yes, since C# 8. You can provide a default method body in an interface. This allows adding new members without breaking existing implementations. The default is only accessible through the interface type, not the class type.

Tags: default interface method • C# 8

Q24. What is explicit interface implementation?

Answer: When a class implements multiple interfaces with the same method signature, use explicit implementation to disambiguate: void IShape.Draw() { ... }. The method is only accessible when the object is referenced as that interface type.

Tags: explicit interface

Interfaces & Design Principles — Intermediate

Q25. What is the difference between composition and inheritance? Which is preferred?

Answer: Inheritance: 'is-a' relationship. Composition: 'has-a' relationship — a class contains an instance of another class. Composition is generally preferred because it is more flexible, avoids tight coupling, and doesn't break encapsulation (favor composition over inheritance principle).

Tags: composition • inheritance • design

Q26. What is the Liskov Substitution Principle (LSP)?

Answer: LSP states that objects of a derived class should be replaceable by objects of the base class without altering program correctness. If Bird has Fly() but Penguin extends Bird and cannot fly, LSP is violated.

Tags: SOLID • LSP

Q27. What is the Open/Closed Principle?

Answer: A class should be open for extension but closed for modification. Add new behavior by extending (via inheritance or composition) rather than modifying existing code. This reduces the risk of

breaking existing functionality.

Tags: SOLID • OCP

Q28. What is the Single Responsibility Principle?

Answer: A class should have only one reason to change — it should do one thing. For example, a Report class should generate a report but not also be responsible for printing or saving it.

Tags: SOLID • SRP

Advanced OOP — Advanced

Q29. What are generics and how do they relate to OOP?

Answer: Generics allow classes and methods to operate on any data type without sacrificing type safety. Example: List. They promote reuse and avoid boxing/unboxing overhead. Combined with constraints (where T : IComparable), they enforce OOP contracts on type parameters.

Tags: generics • type safety

Q30. What is covariance and contravariance in C#?

Answer: Covariance (out) allows a generic type to be used as its base type. Contravariance (in) allows a generic type to be used as its derived type. Example: IEnumerable can be assigned to IEnumerable because IEnumerable is covariant.

Tags: covariance • contravariance • generics

Q31. What is the difference between shallow copy and deep copy?

Answer: Shallow copy duplicates the object but not the objects it references — both share the same nested objects. Deep copy duplicates everything recursively. In C# you can implement deep copy via ICloneable, serialization, or manual copy.

Tags: shallow copy • deep copy • cloning

Q32. What is object finalization and the Dispose pattern?

Answer: The garbage collector calls the finalizer (~ClassName()) to clean up unmanaged resources, but timing is non-deterministic. The IDisposable pattern with a Dispose() method allows deterministic cleanup. Use the using statement to ensure Dispose() is called.

Tags: IDisposable • Dispose • finalizer • GC

Q33. What is the difference between early binding and late binding?

Answer: Early binding resolves method calls at compile time (regular method calls, overloading). Late binding resolves at runtime (virtual methods, dynamic type, reflection). Late binding is more flexible but slower.

Tags: early binding • late binding • dynamic

Q34. What is a design pattern? Name and explain two creational patterns.

Answer: Design patterns are reusable solutions to common design problems. Singleton ensures only one instance of a class exists (private static readonly instance). Factory Method defines an interface for creating objects but lets subclasses decide which class to instantiate.

Tags: design patterns • Singleton • Factory

Q35. What is the Dependency Inversion Principle and how does it connect to interfaces?

Answer: High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces). Instead of a class creating its dependencies directly, they are injected. This is the foundation of Dependency Injection (DI) in ASP.NET Core.

Tags: SOLID • DIP • DI

Tip: Always provide a C# code example when answering in interviews. For SOLID principles, be ready to explain with real-world scenarios.